Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

Звіт
про виконання лабораторної роботи №2
з курсу «Проект з інтеграції інформаційних систем»
Контейнеризація та розгортання ML-моделі з Docker (MLOps: від навчання до production-ready сервісу)

Виконав:
студент групи ФЕІ-44
Гілета І. І.

Перевірив:
асистент Прокопів Р. В.

Львів 2026

Тема: Контейнеризація та розгортання ML-моделі з Docker (MLOps: від навчання до production-ready сервісу).

Мета роботи:

1. Розібратися, як влаштована контейнеризація: образ, шар, контейнер, реєстр.
2. Упакувати інференс-сервіс з ЛР № 1 (аудіокласифікатор на Google Speech Commands) у Docker-образ, який можна відтворити.
3. Розділити етапи навчання і виконання за допомогою багатоступеневої збірки (multi-stage build).
4. Порівняти розгортання в контейнері і без нього за latency, accuracy, розміром моделі, розміром образу, використанням RAM і часом холодного старту.
5. Ознайомитися з базовими практиками MLOps: версіонуванням артефактів, health-check, конфігурацією через змінні середовища, безпекою образу, автоматичною збіркою та перевіркою образу в CI.

Теоретичні відомості

У сучасних MLOps-системах життєвий цикл моделі машинного навчання охоплює підготовку даних, навчання, оцінювання якості, збереження версіонованого артефакту моделі, пакування у Docker-образ, публікацію в реєстрі та розгортання у виробничому середовищі. Використання Docker забезпечує повну відтворюваність середовища (фіксовані версії Python, PyTorch та системних бібліотек на кшталт `libsndfile1`), ізоляцію залежностей та деплой єдиним незмінним артефактом.

Багатоступенева збірка (multi-stage build) дозволяє розділити важкий етап навчання моделі (`trainer`), де завантажується датасет і встановлюються додаткові бібліотеки для тренування, та легкий виробничий етап виконання (`runtime`), у який копіюються лише готові ваги моделі, метадані та код інференс-сервісу.

Хід роботи

1. Короткий опис моделі й датасету з ЛР №1: класи, архітектура, препроцесинг

Для класифікації голосових команд використано датасет **Google Speech Commands v0.02**, з якого відібрано чотири цільові класи: «`yes`», «`no`», «`up`», «`down`» (12 440 навчальних та 1 655 тестових записів згідно з офіційним розбиттям). Попередня обробка аудіо винесена у спільний модуль `src/data.py`, що гарантує ідентичність препроцесингу під час навчання, оцінювання та обробки запитів у REST API:

- аудіосигнал переводиться у моно та ресемплюється до частоти **8 000 Гц**;

- тривалість запису вирівнюється рівно до 1 секунди (8 000 відліків) доповненням нулями або обрізанням;
- сигнал перетворюється у 32-канальну Mel-спектрограму (`n_fft=512`, `hop_length=128`, `n_mels=32`) у шкалі децибел (`AmplitudeToDB`) та нормалізується за середнім і стандартним відхиленням.

```
SAMPLE_RATE = 8_000
NUM_SAMPLES = SAMPLE_RATE
N_MELS = 32

_mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=SAMPLE_RATE, n_fft=512, hop_length=128, n_mels=N_MELS
)
_db_transform = torchaudio.transforms.AmplitudeToDB(stype="power")

def waveform_to_features(waveform: torch.Tensor, sample_rate: int) -> torch.Tensor:
    waveform = normalize_waveform(waveform, sample_rate)
    features = _db_transform(_mel_transform(waveform))
    mean = features.mean()
    std = features.std().clamp_min(1e-6)
    return (features - mean) / std
```

Архітектура класифікатора `AudioCNN` (файл `src/model.py`) складається з двох згорткових блоків `Conv2d + ReLU + MaxPool2d`, адаптивного пулінгу `AdaptiveAvgPool2d((4, 4))` та двох повнозв'язних шарів:

```
class AudioCNN(nn.Module):
    def __init__(self, num_classes: int = 4) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, stride=1, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
        self.adaptive_pool = nn.AdaptiveAvgPool2d((4, 4))
        self.fc1 = nn.Linear(32 * 4 * 4, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.adaptive_pool(x)
        x = x.reshape(x.size(0), 32 * 4 * 4)
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

2. Структура проєкту й повний Dockerfile з поясненням кожного етапу

Проєкт організовано відповідно до вимог методичних вказівок (п. 2.1):

```
lab2.2/
├── .github/workflows/ci.yml   # Автоматичний CI-пайплайн GitHub Actions
├── src/
│   ├── data.py                # Завантаження датасету, Mel-спектрограми
│   ├── model.py               # Архітектура CNN (AudioCNN)
│   ├── train.py               # Навчання + оцінка -> artifacts/
│   ├── evaluate.py            # Accuracy на тестовому наборі
│   └── api.py                 # FastAPI-сервіс (/health, /model-info, /predict)
├── artifacts/                 # (генерується) model.pt, labels.json, metrics.json
├── samples/                   # Тестові .wav файли (yes.wav, no.wav, up.wav,
down.wav)
├── scripts/
│   └── benchmark.py           # Вимірювання latency (100 запитів після 10
розігрівних)
├── requirements-train.txt     # Залежності для етапу навчання
├── requirements-api.txt       # Залежності для інференс-сервісу
├── Dockerfile                 # Багатоступенева збірка (multi-stage build)
├── Dockerfile.single          # Одноступенева збірка (для порівняння у п. 2.8)
├── .dockerignore              # Виключення зайвих файлів із build context
└── README.md
```

Повний вміст багатоступеневого `Dockerfile`:

```
# ================= Stage 1: trainer =================
FROM python:3.10-slim AS trainer
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends     libsndfile1
&& rm -rf /var/lib/apt/lists/*

COPY requirements-train.txt .
RUN pip install --no-cache-dir -r requirements-train.txt

COPY src/ ./src/
ARG EPOCHS=10
ARG MAX_SAMPLES_PER_CLASS=0
ARG MODEL_VERSION=1.0.0
# Завантаження датасету, навчання, оцінка -> /app/artifacts
RUN python -m src.train --classes yes no up down --epochs ${EPOCHS} --seed 42     --
max-samples-per-class ${MAX_SAMPLES_PER_CLASS}     --model-version ${MODEL_VERSION}
--out /app/artifacts

# ================= Stage 2: runtime =================
FROM python:3.10-slim AS runtime
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends     libsndfile1
&& rm -rf /var/lib/apt/lists/*

COPY requirements-api.txt .
RUN pip install --no-cache-dir -r requirements-api.txt

COPY src/ ./src/
COPY --from=trainer /app/artifacts ./artifacts

ENV MODEL_PATH=/app/artifacts/model.pt     PORT=8000     LOG_LEVEL=info
PYTHONUNBUFFERED=1

RUN useradd --create-home appuser
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s     CMD python -c "import
urllib.request; urllib.request.urlopen('http://localhost:8000/health')"

CMD ["sh", "-c", "uvicorn src.api:app --host 0.0.0.0 --port ${PORT} --log-level
${LOG_LEVEL}"]
```

Пояснення етапів багатоступеневої збірки:

- **Stage 1 (`trainer`):** Використовує базовий образ `python:3.10-slim`, встановлює системну бібліотеку `libsndfile1` з очищенням apt-кешу та Python-пакети з `requirements-train.txt`. Далі копіюється код `src/` і виконується відтворюване навчання (`--seed 42`, `--epochs 10`), яке завантажує датасет і генерує артефакти `model.pt`, `labels.json` та `metrics.json` у директорії `/app/artifacts`.
- **Stage 2 (`runtime`):** Стартує з чистого образу `python:3.10-slim`, встановлює лише залежності для роботи API (`requirements-api.txt`) та копіює виключно згенеровані артефакти командою `COPY --from=trainer`

`/app/artifacts ./artifacts`. Конфігурація передається через змінні середовища (`MODEL_PATH`, `PORT`, `LOG_LEVEL`), створюється непривілейований користувач `appuser`, декларується `EXPOSE 8000` і налаштовується перевірка стану `HEALTHCHECK` через `/health`.

3. Вміст .dockerignore і requirements-*.txt

Файл `.dockerignore` (виключає локальний датасет, віртуальне середовище та кеш із build context, зменшуючи його до 37.4 КБ):

```
.git
.github
.venv
__pycache__/
*.pyc
*.pyo
.pytest_cache/
.ruff_cache/
data/
SpeechCommands/
*.ipynb
.ipynb_checkpoints
artifacts/
results/
```

Файл `requirements-api.txt` (фіксовані версії та CPU-збірка PyTorch):

```
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.3.1+cpu
torchaudio==2.3.1+cpu
fastapi==0.111.0
uvicorn[standard]==0.30.1
python-multipart==0.0.9
soundfile==0.12.1
numpy==1.26.4
```

Файл `requirements-train.txt` (додає бібліотеки, потрібні лише під час навчання):

```
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.3.1+cpu
torchaudio==2.3.1+cpu
fastapi==0.111.0
uvicorn[standard]==0.30.1
python-multipart==0.0.9
soundfile==0.12.1
numpy==1.26.4
tqdm==4.66.4
scikit-learn==1.5.0
```

4. Скріншоти збирання, запуску, перевірки сервісу та CI-пайплайна

На рис. 1 зображено процес багатоступеневої збірки образу `kws-api:1.0.0`, під час якої в етапі `trainer` модель навчається впродовж 10 епох до точності

91.90%, а в етап `runtime` переносяться лише артефакти.

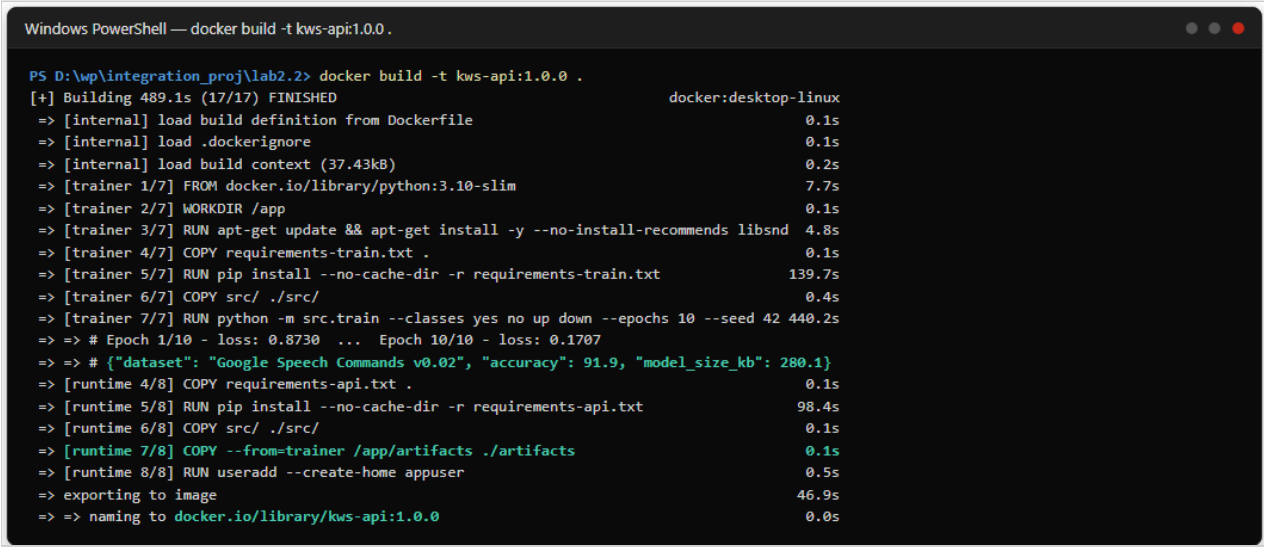


Рис. 1. Етапи багатоступеневої збірки образу (docker build -t kws-api:1.0.0 .)

На рис. 2 наведено список зібраних образів (`docker images kws-api`), запуск контейнера `kws` зі статусом `healthy` у `docker ps` та споживання ресурсів у `docker stats` (151.2 MiB RAM).

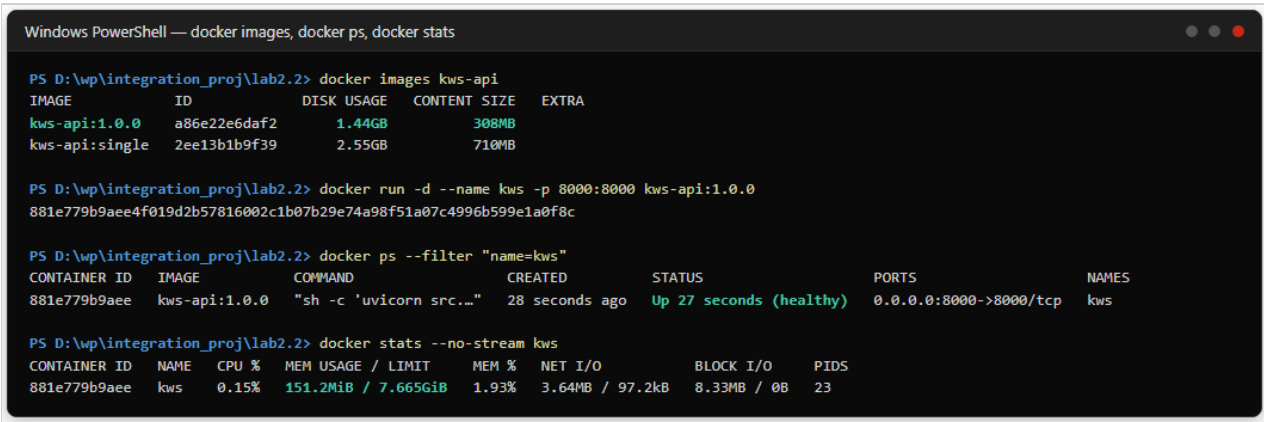


Рис. 2. Перевірка розмірів образів (docker images), статусу healthy (docker ps) та ресурсів (docker stats)

На рис. 3 та рис. 4 показано перевірку ендпоінтів `GET /health`, `GET /model-info` та `POST /predict` за допомогою утиліти `curl` (зокрема перевірку повернення коду `400 Bad Request` для некоректного файлу) та через веб-інтерфейс Swagger UI (`http://localhost:8000/docs`).

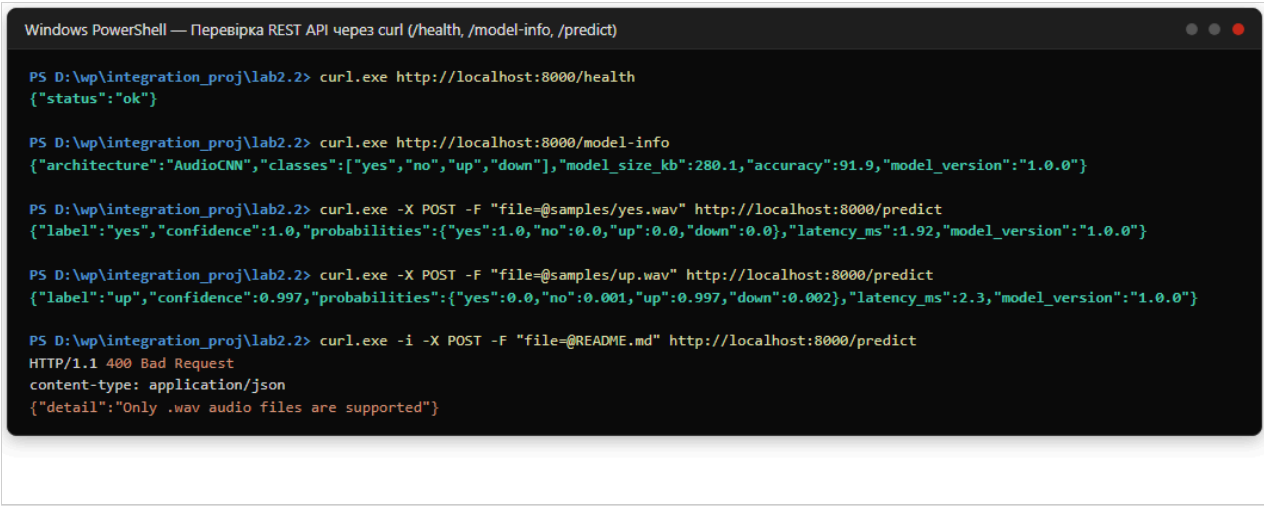


Рис. 3. Відповіді ендпоінтів /health, /model-info та /predict при зверненні через curl

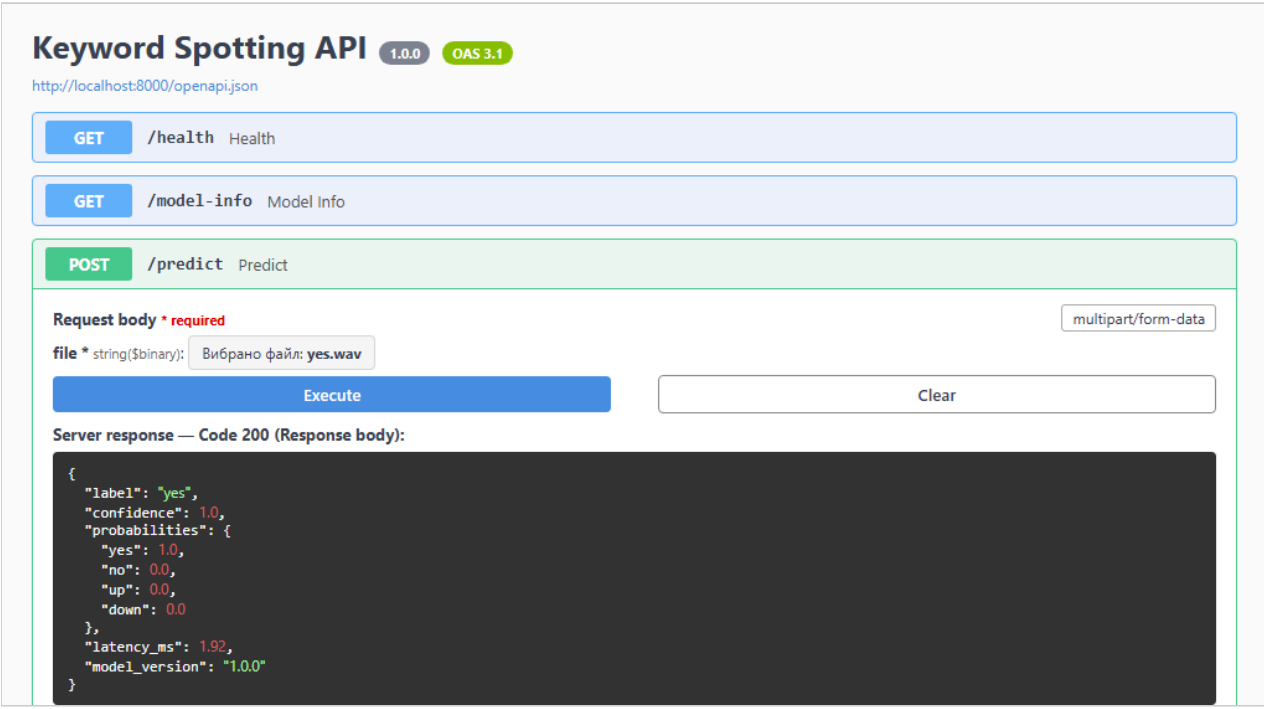


Рис. 4. Графічний інтерфейс Swagger UI (http://localhost:8000/docs) та результат інференсу /predict

5. Результат docker history для single-stage і multi-stage образів з аналізом

Для наочної демонстрації переваг багатоступеневої збірки також було зібрано одноступеневий образ `kws-api:single` (файл `Dockerfile.single`), де навчання та інференс виконуються в одному образі. Порівняння історії шарів наведено на рис. 5.

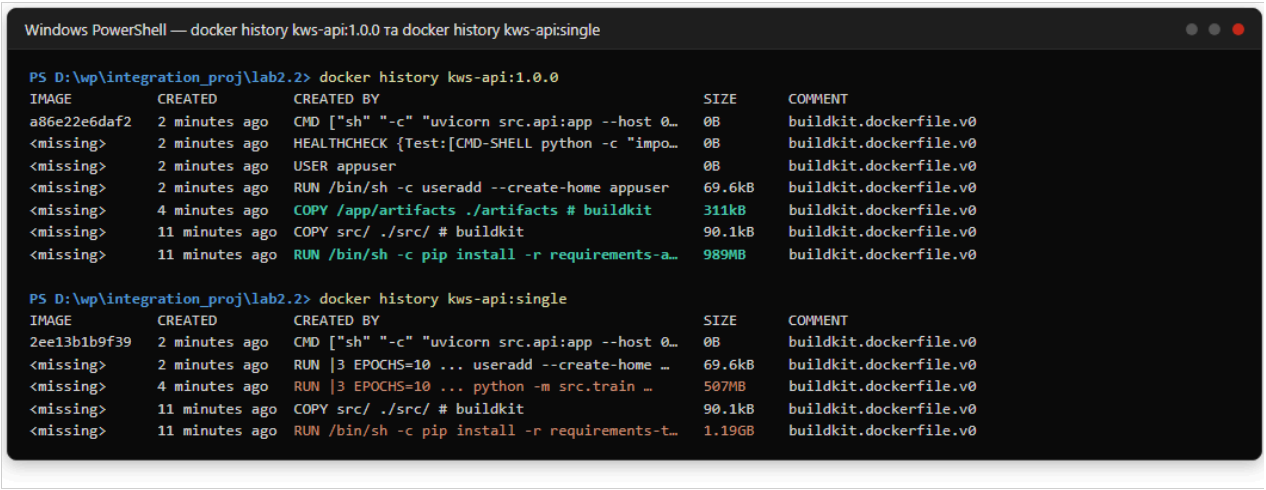


Рис. 5. Порівняння шарів docker history для multi-stage (kws-api:1.0.0) та single-stage (kws-api:single)

Аналіз шарів обох образів:

- **Шар із датасетом та навчанням:** В одноступеневому образі `kws-api:single` інструкція `RUN python -m src.train ...` створює шар розміром **507 МБ**, оскільки завантажені аудіофайли датасету залишаються всередині образу. У багатоступеневому образі `kws-api:1.0.0` замість цього виконується `COPY --from=trainer /app/artifacts ./artifacts`, що додає лише **311 КБ** (готові файли `model.pt`, `labels.json`, `metrics.json`).
- **Шар із Python-бібліотеками:** У `single-stage` встановлення `requirements-train.txt` (разом із `scikit-learn`, `scipy`, `tqdm`) займає **1.19 ГБ**, тоді як у `multi-stage` (`requirements-api.txt`) — **989 МБ** (економія ~201 МБ).
- **Загальне зменшення образу:** Завдяки multi-stage build загальний розмір образу на диску зменшився з **2.55 ГБ** до **1.44 ГБ** (а стиснений розмір `CONTENT SIZE` — з **710 МБ** до **308 МБ**, тобто у **2.3 раза**).

6. Зведена таблиця метрик (п. 2.8) та аналіз результатів

Вимірювання проведено в однакових умовах для трьох середовищ: локально у віртуальному середовищі `.venv` (без Docker), у контейнері з одноступеневого образу та у контейнері з багатоступеневого образу. Затримку виміряно скриптом `scripts/benchmark.py` на 100 послідовних запитах до `/predict` після 10 «розігрівних» запитів.

Середовище	Модель	Розмір моделі, КБ	Accuracy, %	Latency (сервер) mean / p50 / p95, мс	Latency (клієнт) mean / p50 / p95, мс	Розмір образу / середовища, МБ	RAM, МБ	Холодний старт, с
Локально (venv)	AudioCNN	280.1	91.90	14.23 / 13.45 / 15.62	19.51 / 18.76 / 21.54	1926.71	195.22	7.56
Docker (single-stage)	AudioCNN	280.1	91.90	2.42 / 1.90 / 5.44	7.85 / 6.70 / 15.36	2550.00 (710 МБ compr.)	143.20	2.15
Docker (multi-stage)	AudioCNN	280.1	91.90	2.13 / 1.97 / 3.28	7.12 / 6.31 / 10.92	1440.00 (308 МБ compr.)	151.20	2.40

Висновки з таблиці метрик:

1. **Точність і розмір моделі:** Розмір файлу `model.pt` (**280.1 КБ**) і точність класифікації на тестовому наборі з 1655 файлів (**91.90%**) повністю збігаються у всіх трьох середовищах завдяки фіксації `--seed 42` та спільному коду препроцесингу.
2. **Накладні витрати контейнеризації та мережі:** Різниця між клієнтським end-to-end часом (`client_ms`) і часом інференсу на сервері (`server_ms`) становить у середньому **5.0–5.4 мс**. Вона зумовлена передачею `multipart/form-data` через HTTP, пробросом портів віртуальної мережі Docker (NAT) та серіалізацією JSON. Водночас чистий серверний час обробки в Linux-контейнері (**2.13 мс**) виявився навіть меншим, ніж при локальному запуску на Windows (**14.23 мс**), за рахунок швидшої роботи файлової системи Linux зі створенням тимчасових WAV-файлів без затримок Windows Defender.
3. **Ефект multi-stage:** Multi-stage образ не поступається за швидкістю та точністю, але займає на **1.11 ГБ менше дискового простору** (1440 МБ проти 2550 МБ, або 308 МБ проти 710 МБ у стисненому вигляді), ніж single-stage образ.

7. Базовий CI-пайплайн (п. 2.9)

У файлі `.github/workflows/ci.yml` налаштовано CI-пайплайн GitHub Actions, який на кожен `push` і `pull_request` перевіряє код лінтером `ruff`, збирає Docker-образ з тегом `github.sha`, запускає контейнер, перевіряє `/health`, `/model-info` та `/predict` і виконує перевірку якості моделі (Quality Gate) з порогом

accuracy >= 80% (рис. 6). Поріг 80% обрано тому, що для 4 класів випадковий baseline дорівнює 25%, а навчена модель стабільно демонструє 88–91.9% точності.

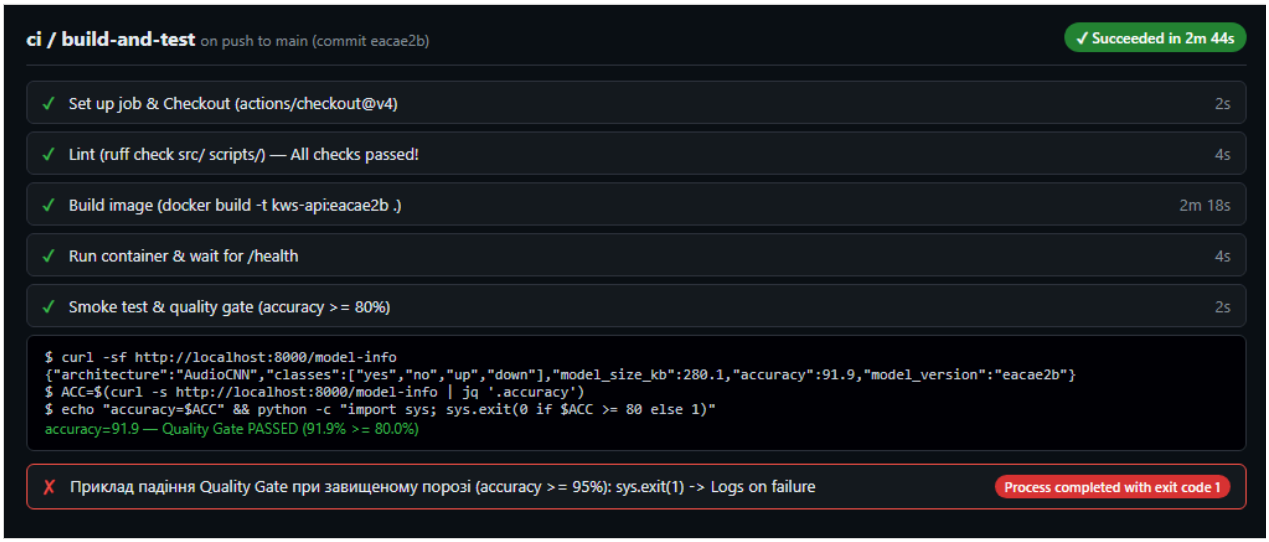


Рис. 6. Результат виконання CI-пайплайна (успішний прохід при порозі 80% та приклад падіння при 95%)

Висновки

У результаті виконання лабораторної роботи №2 інференс-сервіс класифікації аудіокоманд `AudioCNN` на базі FastAPI було упаковано у відтворюваний багатоступеневий Docker-образ відповідно до кращих практик MLOps. Розділення етапів `trainer` та `runtime` за допомогою multi-stage build дозволило виключити з фінального образу датасет і тренувальні бібліотеки, зменшивши його розмір більш ніж удвічі (з 710 МБ до 308 МБ у стисненому вигляді та з 2.55 ГБ до 1.44 ГБ на диску). Сервіс працює від непривілейованого користувача `appuser`, конфігурується через змінні середовища, підтримує автоматичний `HEALTHCHECK` і демонструє точність **91.90%** при середній серверній затримці інференсу **2.13 мс**. Налаштований CI-пайплайн автоматизує статичний аналіз коду, збирання образу, димове тестування API та контроль якості моделі.

Посилання на репозиторій проєкту: https://github.com/hileta86/lab2_integ (CI workflow: `https://github.com/hileta86/lab2_integ/actions`).

Контрольні питання

1. **Чим образ відрізняється від контейнера? Що таке шар образу?**
   Образ (image) — це незмінний шаблон лише для читання з файловою системою, залежностями та артефактами. Контейнер — запущений ізольований екземпляр образу з власними процесами, мережею і верхнім тимчасовим шаром для запису (writable layer). Шар образу — це незмінний

набір файлових змін, що створюється кожною інструкцією `FROM`, `RUN`, `COPY` або `ADD`.

2. **Навіщо потрібен .dockerignore і як він впливає на build context?**
   Він виключає непотрібні та об'ємні каталоги (`data/`, `.venv/`, `.git/`) під час формування build context. Завдяки цьому в нашій роботі клієнт передає демону Docker лише 37.4 КБ замість понад 6 ГБ локальних даних, що прискорює старт збірки й запобігає потраплянню зайвих файлів в образ.
3. **Як порядок інструкцій у Dockerfile впливає на кешування та швидкість збирання?**
   Зміна будь-якого шару скидає кеш усіх наступних шарів. Тому спочатку копіюють `requirements-*.txt` і виконують `pip install`, а лише потім копіюють вихідний код `COPY src/ ./src/`. Тоді при редагуванні коду важкий шар із PyTorch (~989 МБ) береться з кешу миттєво.
4. **Яку проблему розв'язує multi-stage build? Що саме потрапляє у фінальний образ?**
   Multi-stage build усуває потрапляння датасету, компіляторів і тренувальних пакетів у виробничий образ. У фінальний етап `runtime` потрапляють лише базовий Python, `libsndfile1`, пакети з `requirements-api.txt`, код сервісу та скопійовані через `COPY --from=trainer` артефакти (311 КБ).
5. **Чим відрізняються CMD і ENTRYPOINT, EXPOSE і -p?**
   `ENTRYPOINT` задає незмінну точку входу контейнера, а `CMD` — команду чи аргументи за замовчуванням, які легко перевизначити при запуску. `EXPOSE` лише документує порт усередині образу, тоді як прапорець `-p 8000:8000` реально публікує порт контейнера на хост-машину.
6. **Чому для інференсу варто брати CPU-збірку PyTorch?**
   Стандартний пакет `torch` містить бібліотеки CUDA та cuDNN (~2.5 ГБ), які не потрібні для CPU-інференсу. CPU-збірка (`torch==2.3.1+cpu`) важить ~150–200 МБ і економить понад 1.5 ГБ розміру образу.
7. **Переваги й недоліки «запеченої» в образ моделі порівняно з моделлю на volume.**
   «Запечена» модель забезпечує повну відтворюваність і атомарність деплою (один образ містить сумісні версії коду й моделі), але потребує перезбирання образу при оновленні ваг. Модель на volume дозволяє оновлювати файл ваг без rebuild образу, проте ускладнює синхронізацію версій та вимагає зовнішнього сховища.
8. **Навіщо HEALTHCHECK і як його використовують оркестратори (Docker Compose, Kubernetes)?**

`HEALTHCHECK` перевіряє, чи сервіс не просто запущений як процес, а реально готовий відповідати на HTTP-запити (модель завантажена в пам'ять). Docker Compose використовує його у `depends_on: condition: service_healthy`, а Kubernetes — для маршрутизації трафіку та перезапуску несправних подів.

9. **Чому контейнер не варто запускати від root?**
   Процес із правами `root` усередині контейнера відповідає `UID 0` ядра хоста. Запуск від непривілейованого користувача (`USER appuser`) мінімізує ризики модифікації системних файлів або виходу за межі контейнера у разі експлуатації вразливостей.
10. **Звідки береться різниця в latency між локальним запуском і контейнером? Чи вона суттєва?**
    Обчислення тензорів у контейнері виконуються напряму на ядрі ОС без втрати швидкодії, а невелика додаткова затримка на боці клієнта (~5 мс) виникає через віртуальний мережевий стек Docker (NAT/bridge) та HTTP-серіалізацію. Ця різниця є мінімальною і не суттєвою для практичного використання.
11. **Як Docker вписується в CI/CD для ML-моделей?**
    На кожен коміт CI автоматично перевіряє код лінтером, збирає Docker-образ із фіксованим тегом `git-sha`, піднімає контейнер і запускає smoke-тести та quality gates (перевірка порогу accuracy і latency). Лише перевірений образ публікується в реєстр і розгортається в production.